Python Operators

This document explains the main types of operators in Python with explanations and practical code examples.

by: Abdesselam Filali - *infos@filali.net*

Date : *March 25, 2026*

Python Operators

Python operators are used to perform operations on variables and values.

There are different types of operators in Python:

- Arithmetic operators
- Comparison operators
- Logical operators
- Assignment operators
- Bitwise operators
- Membership operators
- Identity operators
- Bitwise Operators

1. Arithmetic Operators

Arithmetic operators in Python are used to perform mathematical operations between variables and values.

Operator	Description	Example
`+`	Adds two operands	`5 + 3`
`-`	Subtracts the second operand from the first	`5 - 3`
`*`	Multiplies two operands	`5 * 3`
`/`	Divides the first operand by the second (returns float)	`5 / 3`
`//`	Divides the first operand by the second (returns integer)	`5 // 3`
`%`	Returns the remainder	`5 % 3`
`**`	Power (exponentiation)	`5 ** 3`

Python Arithmetic Operators - Example

```python
a = 10
b = 3

print(a + b)   # 13
print(a - b)   # 7
print(a * b)   # 30
print(a / b)   # 3.333...
print(a // b)  # 3
print(a % b)   # 1
print(a ** b)  # 1000
```

2. Comparison Operators

Comparison operators in Python are used to compare two values.

Operator	Name	Description	Example
`==`	Equal to	True if both operands are equal	`5 == 3`
`!=`	Not equal to	True if operands are not equal	`5 != 3`
`>`	Greater than	True if left operand is greater	`5 > 3`
`<`	Less than	True if left operand is smaller	`5 < 3`
`>=`	Greater than or equal to	True if left operand is greater or equal	`5 >= 3`
`<=`	Less than or equal to	True if left operand is smaller or equal	`5 <= 3`

Python Comparison Operators - Example

```python
x = 5
y = 10

print(x == y)  # False
print(x != y)  # True
print(x > y)   # False
print(x < y)   # True
print(x >= y)  # False
print(x <= y)  # True
```

3. Logical Operators

Logical operators in Python are used to combine conditional statements.

Operator	Description	Example
`and`	True if both statements are true	`True and False`
`or`	True if at least one statement is true	`True or False`
`not`	Reverses the result (True becomes False and vice versa)	`not True`

Python Logical Operators - Example

```python
text = "python"
print("p" in text)      # True
print("z" in text)      # False
print("z" not in text)  # True
```

4. Assignment Operators

Are used to assign values to variables and can combine an operation with assignment.

Operator	Name	Example	Equivalent
`=`	Assign	`a = 5`	—
`+=`	Add and assign	`a += 3`	`a = a + 3`
`-=`	Subtract and assign	`a -= 3`	`a = a - 3`
`*=`	Multiply and assign	`a *= 3`	`a = a * 3`
`/=`	Divide and assign	`a /= 3`	`a = a / 3`
`//=`	Floor divide and assign	`a //= 3`	`a = a // 3`
`%=`	Modulus and assign	`a %= 3`	`a = a % 3`
`**=`	Exponentiate and assign	`a **= 3`	`a = a ** 3`

Python Assignment Operators - Example

```python
a = 10
a += 5
a -= 2
a *= 3
a = 4
print(a)
```

6. Membership Operators

Membership operators are used to test if a sequence is present in an object.

Operator	Description	Example
`in`	Returns True if a sequence is present in the object	`a in b`
`not in`	Returns True if a sequence is not present in the object	`a not in b`

Python Membership Operators - Example

```
text = "python"
print("p" in text)      # True
print("z" in text)      # False
print("z" not in text)  # True
```

7. Identity Operators

Identity operators are used to compare objects, not if they are equal, but if they are the same object in memory.

Operator	Description	Example
`is`	Returns True if both variables are the same object	`a is b`
`is not`	Returns True if both variables are not the same object	`a is not b`

Python Identity Operators - Example

```python
a = [1, 2, 3]
b = a
c = a.copy()

print(a is b)       # True
print(a is c)       # False
print(a is not c)   # True
print(a == c)       # True
```

8. Bitwise Operators

Bitwise operators are used to perform bit-level operations on integers.

Operator	Description	Example
`&`	Bitwise AND	`5 & 3`
`|`	Bitwise OR	`5 | 3`
`^`	Bitwise XOR	`5 ^ 3`
`~`	Bitwise NOT	`~5`
`<<`	Bitwise left shift	`5 << 1`
`>>`	Bitwise right shift	`5 >> 1`

Python Bitwise Operators - Example

```python
a = 5  # 0101 in binary
b = 3  # 0011 in binary

print(a & b)  # 1  (0001 in binary)
print(a | b)  # 7  (0111 in binary)
print(a ^ b)  # 6  (0110 in binary)
print(~a)     # -6 (in two's complement)
print(a << 1) # 10 (1010 in binary)
print(a >> 1) # 2  (0010 in binary)
```